

Numerical Analysis of the SIR Epidemic Model: When Does the ODE Solver Matter for Parameter Estimation?

CSE 402 Final Project Report — Section B, Group 02

Saif Uz Zaman
Bangladesh University of Engineering
and Technology
Dept. of CSE • Student ID 2105088
Dhaka, Bangladesh

Sayaad Muzahid Masfi
Bangladesh University of Engineering
and Technology
Dept. of CSE • Student ID 2105066
Dhaka, Bangladesh

Ibtida bin Ahmed
Bangladesh University of Engineering
and Technology
Dept. of CSE • Student ID 2105061
Dhaka, Bangladesh

Sakif Naieb Raiyan
Bangladesh University of Engineering
and Technology
Dept. of CSE • Student ID 2105065
Dhaka, Bangladesh

Aurchi Chowdhury
Bangladesh University of Engineering
and Technology
Dept. of CSE • Student ID 2105083
Dhaka, Bangladesh

SOURCE CODE, DATA AND EXPERIMENTS • PUBLIC GITHUB REPOSITORY

<https://github.com/g-schatten/CSE402-Project>

Abstract

Fitting the SIR epidemic model to case data is the standard way to estimate the transmission rate β , the recovery rate γ and the reproduction number $\mathcal{R}_0$. Our base paper, Capaldi et al. (2012), studies this inverse problem in depth but treats the ODE solver as a black box. We make the solver and its step size h the central experimental variables. From scratch, we implement six ODE solvers (Euler, Heun, RK4, Dormand–Prince RK45, backward Euler, trapezoidal), Levenberg–Marquardt estimation with forward sensitivity equations and LU decomposition, power-method eigenvalue routines, and four uncertainty-quantification methods, including Monte Carlo simulation and Metropolis–Hastings sampling. Every accuracy figure is measured against a solver-free gold standard built from the model’s exact phase-plane reduction. In 14 reproducible experiments the solvers reach their theoretical orders (observed 1.03, 1.94 and 3.94 for Euler, Heun and RK4). The bias they cause in the fitted β follows the same order: a five-fold smaller step reduces it by factors of 5.0, 24.2 and 608. This bias competes with the spread caused by noise, and the two are equal at a crossover noise level σ^* . For Euler, σ^* is 5.8% of peak prevalence at $h = 0.1$ and 15.6% at $h = 0.25$, as a simple scaling law $\sigma^* \propto h^p$ predicts. Below σ^* the solver dominates the estimation error; above it, noise does. We also reproduce the base paper’s identifiability findings, and our fit to the 1666 Eyam plague matches the original study’s $\mathcal{R}_0$ within 1%.

Keywords

SIR model, ODE solvers, convergence order, parameter estimation, Levenberg–Marquardt, Monte Carlo simulation, uncertainty quantification

1 Introduction & Base Paper Review

1.1 Background

The SIR model of Kermack and McKendrick [16] divides a closed population of N people into susceptible (S), infectious (I) and recovered (R) compartments:

$$\dot{S} = -\frac{\beta SI}{N}, \quad \dot{I} = \frac{\beta SI}{N} - \gamma I, \quad \dot{R} = \gamma I. \quad (1)$$

The transmission rate β and the recovery rate γ (mean infectious period $1/\gamma$) set the basic reproduction number $\mathcal{R}_0 = \beta/\gamma$. Prevalence peaks when S first falls to $N/\mathcal{R}_0$ [12]. In practice β and γ are unknown, and they are estimated by fitting (1) to case counts, usually by least squares.

Every evaluation of that least-squares cost needs a numerical solution of (1), and that solution carries truncation error that depends on the integrator and its step size h . The fitted parameters are therefore a property of the data, the model *and* the numerical method. A coarse forward-Euler solve and an RK4 solve of the same equations give visibly different curves (Figure 1), so they must also give different estimates $(\hat{\beta}, \hat{\gamma})$. This project asks how large that effect is, and when it matters.

1.2 Base paper: Capaldi et al. (2012)

Our base paper is Capaldi, Behrend, Berman, Smith, Wright and Lloyd, “Parameter estimation and uncertainty quantification for an epidemic model”, Mathematical Biosciences and Engineering 9(3), 2012 [2]. It is an inverse-problem study. The authors ask how well (β, γ) and $\mathcal{R}_0$ can be recovered from an outbreak time series, and how the answer depends on the way the data were collected.

Statistical model. Observations are $Y_i = I(t_i; \theta_0) + I(t_i; \theta_0)^\xi e_i$, where the e_i are i.i.d. with mean zero and variance σ_0^2 . The exponent $\xi \in \{0, \frac{1}{2}, 1\}$ selects absolute, Poisson-like or relative noise, and

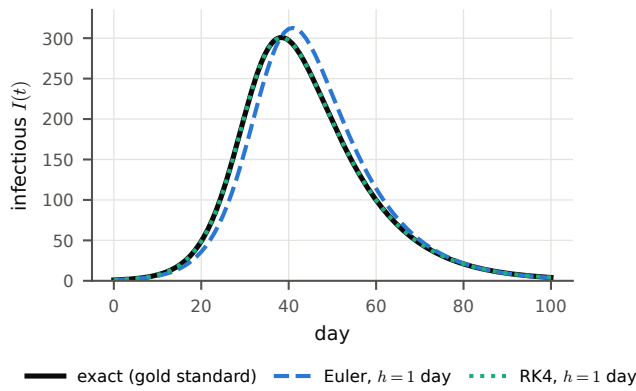


Figure 1: Same parameters $(\beta, \gamma) = (0.3, 0.1)$, different solvers. With a one-day step, forward Euler overestimates the peak by 12 people and places it 2.6 days late; RK4 is indistinguishable from the exact, solver-free curve.

incidence data are treated the same way. Parameters are estimated by ordinary or generalized least squares. Uncertainty comes from asymptotic theory, $\Sigma \approx \sigma^2(\chi^\top W \chi)^{-1}$, where χ is the sensitivity matrix $\partial I(t_i)/\partial \theta_j$. The sensitivities are obtained by solving the forward sensitivity equations together with the model.

Design. The data are synthetic: $N = 10,000$, $I_0 = 100$ and $\gamma = 1$, with $\mathcal{R}_0 \in \{1.2, 3, 10\}$. The model is solved with MATLAB's ode45, and the output is sampled until I falls back to I_0 .

Findings.

- (1) The estimates $\hat{\beta}$ and $\hat{\gamma}$ are correlated, and the sign and size of the correlation depend on $\mathcal{R}_0$: $\rho = 0.98, 0.11$ and -0.31 for $\mathcal{R}_0 = 1.2, 3$ and 10 . Near threshold the cost contours are long ellipses along $\beta = \mathcal{R}_0 \gamma$, so the ratio $\mathcal{R}_0$ is easier to estimate than either parameter on its own.
- (2) Most of the information lies between the start of the outbreak and just after its peak. The standard errors fall steeply over that interval and only slowly afterwards.
- (3) Fitting initial conditions as well destroys identifiability: the condition number of $\chi^\top W \chi$ rises from about 5 for (β, γ) to 10^6 – 10^{10} once I_0 or S_0 is also fitted.

What the base paper leaves open. Throughout the paper the integrator is a black box. The solver is never varied, no convergence order is reported, and the error in $\hat{\theta}$ caused by the discretization is never separated from the error caused by noise. The paper implicitly assumes that the solver is accurate enough for the statistical model to be the whole story.

1.3 Objective and research questions

Our project proposal asked whether *the choice and accuracy of the ODE solver materially affect the parameters we recover, especially once real-world imperfections such as noise and sparse sampling are introduced*. We therefore make the solver and its step size the independent variables, and cross them with noise level, sampling and the true (β, γ) . The proposal set three research questions, and our project plan added a fourth as an extension:

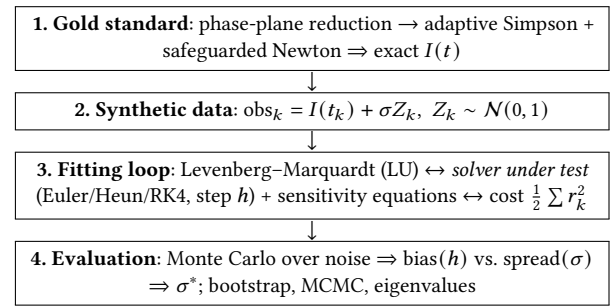


Figure 2: The solver-in-the-loop estimation pipeline.

RQ1 – Solver comparison. How do forward Euler, Heun and RK4 differ in accuracy, convergence order and numerical stability on the SIR system?

RQ2 – Parameter recovery. How does the choice of solver propagate into the accuracy of (β, γ) recovered by least squares on synthetic data?

RQ3 – Noise and sampling robustness. How robust is recovery to observation noise, reduced sampling frequency and initial conditions, and does that robustness hold across a range of true (β, γ) ?

RQ4 – Extension. Do the findings survive harder models (SEIR, SIRS, SIR with vaccination) and a real data set (Eyam, 1666)?

RQ2 and RQ3 meet in a single quantity, the *crossover noise level* σ^* at which the bias caused by the solver equals the spread caused by noise.

1.4 Contributions

- A **solver-free gold standard** for SIR, built from the exact phase-plane reduction [31], adaptive Simpson quadrature and safeguarded Newton iteration. Every accuracy figure is measured against it, never against “a finer numerical solve”.
- A from-scratch numerical library, `sir1ab`: 6 ODE solvers, LU and QR factorization, power-method eigenvalue routines, 5 optimizers and 4 uncertainty-quantification methods. A test enforces that it never imports SciPy.
- Evidence that solver bias in $\hat{\beta}$ scales as h^p with the solver’s own order p , a measured crossover $\sigma^*(h)$, and a scaling law that predicts it.
- 14 seeded, reproducible experiments and an interactive web dashboard, all public at <https://github.com/g-schatten/CSE402-Project>.

2 Methodology

Figure 2 summarizes the pipeline. We choose a true $\theta = (\beta, \gamma)$ and compute the exact epidemic curve without any ODE solver (§2.1). We add controlled noise (§2.5) and fit θ back with a least-squares loop (§2.3) whose forward model runs a chosen solver at a chosen step size (§2.2). Because the truth is known, the error in $\hat{\theta}$ can be split into a solver part and a noise part (§2.6).

2.1 A solver-free gold standard

Dividing $\dot{S}$ by $\dot{R}$ in (1) eliminates time and gives $dS/dR = -(\beta/\gamma N)S$. Integrating once,

$$S = S_0 e^{-kR}, \quad k = \frac{\beta}{\gamma N}, \quad (2)$$

with $R(0) = 0$. Substituting (2) into $\dot{R} = \gamma(N - S - R)$ leaves one separable scalar ODE, whose solution is the quadrature [31]

$$t(R) = \int_0^R \frac{dr}{\gamma(N - r - S_0 e^{-kr})}. \quad (3)$$

Since $\dot{R} = \gamma I > 0$, $t(R)$ is strictly increasing and can be inverted. Our implementation (`core/sirlab/reference.py`) works as follows:

- **Quadrature.** (3) is evaluated by *adaptive Simpson quadrature* with tolerance 10^{-13} . Each accepted panel adds the Richardson correction $(S_2 - S_1)/15$, and a panel is also accepted once its error estimate reaches the round-off floor, so the scheme never chases noise.
- **Table.** The integrand blows up as $R \rightarrow R_\infty$, so t is tabulated once per parameter set on 2,000 Chebyshev-clustered cells, stopping just short of the asymptote.
- **Inversion.** A query time t is bracketed by binary search in the table, and $R(t)$ is then found by *safeguarded Newton–Raphson*, the `rtsafe` scheme [30]. A Newton step is taken only if it stays inside the sign-change bracket and at least halves it; otherwise the method bisection.

The safeguard matters. On the final-size equation

$$\ln \frac{S_0}{S_\infty} = \mathcal{R}_0 \frac{N - S_\infty}{N}, \quad (4)$$

with $N = 1000$ and $\mathcal{R}_0 = 3$, plain Newton started at $S = 500$ converges to the spurious root $S_\infty = 1000.5 > N$. The safeguarded version returns the physical root $S_\infty = 59.448$. The peak height has the closed form $I_{\max} = N - \frac{N}{\mathcal{R}_0} (1 + \ln \frac{\mathcal{R}_0 S_0}{N})$ (with $S_0 + I_0 = N$), and the peak time is $t(R)$ evaluated where $S = N/\mathcal{R}_0$. The gold standard is cross-checked against SciPy’s DOP853 integrator in the test suite.

Equation (2) also gives a *phase invariant* $Q = \ln S + kR$, which is constant along every true trajectory. The obvious check $S + I + R = N$ is useless here, because the right-hand sides of (1) sum to zero identically: every Runge–Kutta method keeps the total exact to round-off, however wrong its solution. Drift in Q has no such loophole.

2.2 ODE solvers

All solvers share one interface (`solvers.integrate`). They return states on a grid, which are evaluated at the observation times by piecewise cubic Hermite interpolation.

- **Forward Euler**, $y_{n+1} = y_n + hf(t_n, y_n)$: order 1, one f -evaluation per step.
- **Heun** (explicit trapezoid): an Euler predictor $\tilde{y}$ followed by the corrector $y_{n+1} = y_n + \frac{h}{2} [f(t_n, y_n) + f(t_{n+1}, \tilde{y})]$: order 2, two evaluations.
- **Classical RK4**, with four slopes, $y_{n+1} = y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4)$: order 4, four evaluations.

- **Dormand–Prince RK45** [4]: an adaptive 5(4) pair with a PI step-size controller. It is the reference solution for model variants that have no closed form.
- **Backward Euler** and the **trapezoidal rule** (A-stable, implicit). Each step solves $G(y) = y - y_n - hf(t_{n+1}, y) = 0$, or its trapezoidal analogue, by *Newton’s method for systems*, $(I - hJ_f)\delta = -G$, with our own LU factorization.

On the test equation $y' = \lambda y$, an explicit RK step multiplies y by its stability polynomial: $1 + z$ (Euler), $1 + z + z^2/2$ (Heun) and $\sum_{j \leq 4} z^j/j!$ (RK4), with $z = h\lambda$. On the negative real axis these require $|z| \leq 2$, 2 and 2.785 respectively [10].

2.3 Parameter estimation

Given observations obs_k at times t_k , we minimize

$$J(\theta) = \frac{1}{2} \sum_k r_k(\theta)^2, \quad r_k = I_h(t_k; \theta) - \text{obs}_k, \quad (5)$$

where I_h is the prevalence computed by the *solver under test* at step h . The production estimator is **Levenberg–Marquardt** [20, 22] (`estimate/lm.py`). Each iteration solves

$$(J_r^T J_r + \lambda \text{diag}(J_r^T J_r))\delta = -J_r^T r \quad (6)$$

by LU decomposition. An accepted step (lower cost) divides λ by 10, and a rejected one multiplies it by 10, so the method moves between Gauss–Newton ($\lambda \rightarrow 0$) and scaled gradient descent (large λ). The constraint $\beta, \gamma > 0$ is enforced by optimizing $u = \ln \theta$. The Jacobian $J_r = \partial r / \partial \theta$ does not come from finite differences. Instead we integrate the *forward sensitivity equations*

$$\dot{s}_j = \frac{\partial f}{\partial y} s_j + \frac{\partial f}{\partial \theta_j}, \quad s_j(0) = 0, \quad (7)$$

as a 9-variable augmented system, with the *same* solver and step as the trajectory. Solver error therefore enters the gradient exactly as it enters the curve. The asymptotic covariance is $\widehat{\text{Cov}}(\hat{\theta}) = \hat{\sigma}^2 (J_r^T J_r)^{-1}$, with $\hat{\sigma}^2 = 2J/(n - 2)$. For comparison we also implemented Gauss–Newton (in linear and log coordinates, with an optional Householder–QR solve), Nelder–Mead, golden-section coordinate descent and grid search.

2.4 Linear algebra and eigenvalues

LU decomposition (Doolittle, partial pivoting, `linalg/lu.py`) solves every LM step, every implicit Newton step and every covariance inverse.

Eigenvalues come from our implementation of the course lecturer’s methods [3] (`linalg/eigen.py`):

- the **power method**, where $y = Ax_k$ is normalized by its largest entry and that entry is the eigenvalue estimate;
- the **inverse power method**, the same loop on A^{-1} , with one LU factorization;
- **Hotelling deflation**, $A \leftarrow A - \lambda_1 \hat{v}_1 \hat{v}_1^T$, for the remaining eigenpairs of a symmetric matrix.

These routines give the axes of the confidence ellipse, the direction of the cost valley, the condition number $\kappa_2(J_r) = \sqrt{\lambda_{\max}/\lambda_{\min}}$ of $J_r^T J_r$, and the stability metric $h \rho(\partial f / \partial y)$. The tests reproduce the lecturer’s worked tables and agree with LAPACK.

The SIR Jacobian breaks the power method’s assumption of a single dominant eigenvalue. Its non-zero eigenvalues are those of

$\begin{bmatrix} -a & -b \\ a & b-\gamma \end{bmatrix}$, where $a = \beta I/N$ and $b = \beta S/N$. At the peak $b = \gamma$, giving $\lambda = \frac{1}{2}(-a \pm \sqrt{a^2 - 4a\gamma})$, a complex pair of equal modulus whenever $a < 4\gamma$. There the plain iteration oscillates forever. Our code detects the stall and orthonormalizes $\{x, Ax\}$ into Q . It then solves the 2×2 characteristic equation of $H = Q^T A Q$, and accepts the result once $AQ = QH$ holds to 10^{-10} .

2.5 Random numbers and stochastic simulation

Noise model. Synthetic observations are

$$\text{obs}_k = I(t_k) + \sigma Z_k, \quad Z_k \stackrel{iid}{\sim} \mathcal{N}(0, 1), \quad \sigma = \sigma_{\text{frac}} \max_k I(t_k), \quad (8)$$

which is Capaldi's $\xi = 0$ case. Noise levels are quoted as σ_{frac} , a fraction of peak prevalence.

Generator. All draws come from NumPy's `default_rng`, the PCG64 generator [11, 28]. PCG64 is a 128-bit linear congruential recurrence $X_{i+1} = (aX_i + c) \bmod 2^{128}$, followed by an output permutation that hides the weak low bits of a power-of-two LCG. NumPy draws normals by the ziggurat method [24], bounded integers (bootstrap indices) by Lemire's unbiased method [19], and gamma variates (the MCMC noise variance) by Marsaglia–Tsang [23]. We did not write the generator ourselves.

Reproducible parallel streams. Replicates run in parallel worker processes, and each replicate gets its own generator, seeded by

$$s = (10^7 s_0 + \text{CRC32}(\text{cell key} | r)) \bmod (2^{31} - 1), \quad (9)$$

where s_0 is the experiment's base seed and the key names the experimental cell and replicate r . Results are therefore identical for any number of workers and any scheduling order.

Monte Carlo and uncertainty quantification. Monte Carlo replication is the main evaluation tool. For each cell we generate many independent data sets from the known truth, refit each one, and report the bias, standard deviation and RMSE of $\hat{\theta}$. On a single data set we compare four uncertainty-quantification methods (uq/):

- (1) the asymptotic ellipse from $\widehat{\text{Cov}}$;
- (2) a residual bootstrap [5];
- (3) Monte Carlo over fresh noise, the only method that measures the true bias;
- (4) adaptive Metropolis–Hastings [9] in $\ln \theta$, with a Gibbs update for σ^2 , four chains, the Gelman–Rubin $\hat{R}$ [7] and the effective sample size.

A profile likelihood (golden-section minimization over γ at each fixed β) exposes non-identifiability.

2.6 Bias–variance split and the crossover σ^*

For a fitting solver of order p at step h and noise level σ , the error in $\hat{\beta}$ decomposes as

$$\text{MSE}(\hat{\beta}) = \underbrace{b(h)^2}_{\text{solver}} + \underbrace{s(\sigma, h)^2}_{\text{noise}}. \quad (10)$$

We measure $b(h)$ *deterministically* by fitting noise-free gold-standard data, so that any deviation from β_{true} can only come from truncation error. We measure $s(\sigma, h)$ as the Monte Carlo standard deviation over noisy replicates. The **crossover** $\sigma^*(h)$ solves $s(\sigma^*, h) = |b(h)|$.

We locate it by linear interpolation between the two tested noise levels that bracket it. Below σ^* the solver dominates the error budget; above it, noise does.

A simple scaling argument predicts how σ^* behaves. Truncation error gives $b(h) \approx Ch^p$. For small noise the least-squares estimator is locally linear in the data, so $s \approx K\sigma$. Together,

$$\sigma^*(h) \approx \frac{C}{K} h^p, \quad (11)$$

so for Euler ($p = 1$) σ^* should grow linearly with h .

2.7 Numerical methods used

Table 1 lists every numerical method in the project and the place where it does real work. All of them are our own code, except the random number generator.

3 Implementation & Experiments

3.1 Software and libraries

The code is public at <https://github.com/g-schatten/CSE402-Project> [40]. Its main parts are:

- `core/sirlab/`: the numerical library, about 3,100 lines of Python 3.11 [32] on NumPy [11] arrays. It contains `models` (SIR, SEIR, SIRS and SIR-V, with analytic Jacobians), `solvers`, `linalg`, `quadrature`, `roots`, `estimate` and `uq`, plus `reference.py` (the gold standard) and `sensitivity.py`.
- `experiments/` and `configs/`: one script and one YAML file (read with PyYAML [33]) per experiment, run through a common registry and runner.
- `tests/`: 82 pytest [18] tests, all passing, two of them property-based with Hypothesis [21]. They check:
 - LU, QR and the eigenpairs against SciPy and LAPACK [37];
 - the gold standard against SciPy's DOP853 integrator;
 - the model Jacobians against complex-step derivatives;
 - the observed convergence orders, and exact recovery by every optimizer;
 - two “hygiene” rules: the core never imports SciPy or matplotlib, and nothing outside the tests calls NumPy's eigen- or SVD solvers.
- `notebooks/`: one Jupyter [17] notebook per research question, reading the same result files (one table uses pandas [25]).
- `web/`: a 10-page dashboard in TypeScript [27] with React [26], React Router [35], Zustand [29] and Framer Motion [6], charts on D3 scales [1], equations in KaTeX [15], built with Vite [38]. It ships a TypeScript copy of the models, solvers and gold standard, so the Model Lab, Solver Arena and Stability pages recompute in the browser. The other pages load the precomputed results. A Vitest [39] parity test (9 cases, passing) checks the TypeScript code against Python-generated fixtures: solver states to 10^{-9} and the gold standard to 10^{-6} .

Each run records its config hash, git commit, library version, seed, wall time and host in `results/manifest.json`, and a run is skipped when its config is unchanged. The figures in this report are drawn from the result files by `report/make_report_figures.py` with matplotlib [14], and the report uses the ACM `acmart` template [36].

Table 1: Numerical methods implemented and their role (paths relative to `core/sirlab/`). [†]Provided by NumPy (§2.5).

Method	Role in the pipeline	Code
Errors and root finding		
Truncation / round-off error	Observed order $\hat{p}$; $S + I + R$ exact to round-off; bias $\propto h^p$	diagnostics.py
Bisection + Newton–Raphson	Safeguarded Newton inverts $t(R)$ and solves the final-size equation (4)	roots/newton.py
Newton’s method for systems	Each backward-Euler / trapezoidal step, $(I - hJ_f)\delta = -G$	solvers/implicit.py
Linear systems and eigenvalues		
LU with partial pivoting	Every LM step (6), implicit Newton step, covariance inverse and inverse-power step	linalg/lu.py
Householder QR	Optional least-squares solve that avoids $\kappa(J)^2$	linalg/qr.py
Power / inverse power, deflation	Ellipse axes, cost-valley direction, $\kappa_2(J_f)$, stability metric $h\rho(J)$	linalg/eigen.py
Optimization and curve fitting		
Nonlinear least squares	The cost (5) in every fit; linear least squares on $(\ln h, \ln e)$ gives $\hat{p}$	estimate/, diagnostics.py
Gradient / Newton-type methods	Levenberg–Marquardt (production), Gauss–Newton	estimate/lm.py, gaussnewton.py
Golden section, Nelder–Mead	Coordinate descent and profile likelihood; derivative-free baseline	estimate/golden.py, estimate/neldermead.py
Interpolation and integration		
Cubic Hermite / linear interp.	Solver output at the observation times; locating σ^*	solvers/base.py
Adaptive Simpson + Richardson	The gold-standard integral (3)	quadrature/adaptive.py
Ordinary differential equations		
Runge–Kutta, adaptive, implicit	Euler, Heun, RK4, Dormand–Prince RK45, backward Euler, trapezoidal	solvers/
Simulation		
Random numbers [†]	Noise, random starts, resampling; per-replicate seeds (9)	observe.py, experiments/
Monte Carlo, bootstrap, MCMC	Bias, spread and RMSE of $\hat{\theta}$; confidence regions	uq/

3.2 Simulation setup and parameter choices

Unless stated otherwise, every experiment uses the following setup (from `configs/`):

- **Population:** $N = 1000$, $I_0 = 1$, $S_0 = 999$ and $R(0) = 0$.
- **Reference regime:** $(\beta, \gamma) = (0.3, 0.1)$, so $\mathcal{R}_0 = 3$. The gold standard gives a peak of $I_{\max} = 300.80$ at $t_{\text{peak}} = 38.36$ days and a final size of $S_{\infty} = 59.45$.
- **Other regimes:** $\mathcal{R}_0 = 1.5$ and 6 in the convergence study, $(0.6, 0.15)$ in the recovery study, and a 15×15 grid of (β, γ) in the stability study.
- **Observations:** daily prevalence on days $1, \dots, 60$ ($\Delta t = 1$), well past the peak.
- **Estimator:** Levenberg–Marquardt in $\ln \theta$, with $\lambda_0 = 10^{-2}$, relative cost tolerance 10^{-10} and at most 60 iterations. Each replicate starts from a random point $\theta_0 = \theta_{\text{true}} \odot (U_1, U_2)$, where $U_j \sim U(0.6, 1.6)$, or $U(0.7, 1.4)$ in E10 and E13, so a bias can never be an artifact of starting at the truth.

Step sizes from 0.05 to 0.5 day bracket typical practice. Noise levels $\sigma_{\text{frac}} \in [0.005, 0.2]$ run from near-perfect surveillance to very noisy data.

3.3 Experiments

The repository defines 14 experiments, E01–E14 (`experiments/registry.py`). Table 2 lists them by research question, with their registry names and wall-clock times. Each config defines a quick profile and a full profile at the scale of the original plan (for example about 1.9×10^6 fits for E07). The quick profile runs all 14 experiments in 1,014 s (about 17 minutes) with 8 worker processes on a laptop. The solver studies take seconds; the replicated fitting experiments take 1–5 minutes each. *All numbers in this report come from the quick profile.*

3.4 Datasets and benchmarks

Synthetic data. The recovery, noise, sampling, initial-condition and crossover experiments (E07–E10, E13) generate data from the

gold standard, so the data contain no discretization error at all. The landscape, optimizer and UQ experiments (E05, E06, E11) use an RK4 solve at $h = 0.1$, whose error ($\sim 10^{-6}$) is far below every noise level used. Generating data from the model being fitted is deliberate: it isolates solver and noise error from model misspecification, and it is the design Capaldi et al. use.

Real data. The real-data experiment (E14) uses the 1666 plague outbreak in Eyam, Derbyshire, a village that isolated itself during the outbreak. Raggett [34] reconstructed the susceptible and infective counts from the parish death records, at 0, 0.5, 1, 1.5, 2, 2.5, 3 and 4 months after 18 June 1666 (the last on 20 October). We use his table as reproduced, identically, by Ho et al. [13] and by Golightly and Sherlock [8] (`data/raw/eyam_1666.csv`). We convert months to days ($\times 365.25/12$) and fit the 8 susceptible counts, with $N = 261$ ($S + I$ on 18 June) and $I_0 = 7$.

Benchmarks. The benchmarks are:

- the gold standard, with the closed forms for $I_{\max}$ and S_{∞} , for solver accuracy;
- textbook orders and stability limits, for RQ1;
- Capaldi et al.’s identifiability results, for RQ3;
- Raggett’s own estimate for the Eyam outbreak, for RQ4.

3.5 One fit, traced through the code

To show where each method runs, we instrumented a single fit from the crossover experiment (E13): Euler at $h = 0.1$, $\sigma_{\text{frac}} = 0.05$, 60 daily counts, replicate 0. Table 3 gives the call counts. This one fit needs 19 ODE solves, 6 of them on the 9-variable sensitivity system. The experiment performs 459 such fits.

4 Results & Discussion

All numbers come from the quick profile, and rerunning the pipeline reproduces them exactly.

Table 2: The experiment registry: quick-profile sizes and wall-clock times (8 worker processes). “Fits” are complete Levenberg–Marquardt estimations.

ID	Registry name	RQ	What it measures	Design (quick profile)	Time (s)
E01	Convergence & order	1	Observed order vs. the gold standard	5 solvers $\times h \in \{1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}\} \times 3$ regimes ($\mathcal{R}_0 = 1.5, 3, 6$), $t \in [0, 40]$	1.1
E02	Invariant drift	1	Mass error vs. phase-invariant drift	Euler/Heun/RK4 $\times h \in \{2, 0.5, 0.1\}$, $t \in [0, 60]$	0.1
E03	Stability frontier	1	Largest stable h ; $h\rho(J)$ there	15×15 (β, γ) grid $\times 3$ solvers $\times h \in \{0.5, \dots, 16\}$	9.0
E04	Structural accuracy	1	Errors in peak height/time, final size	3 solvers $\times h \in \{2, 1, 0.5, 0.25\}$	0.2
E05	Cost landscape	3	$J(\beta, \gamma)$, valley axis, $\kappa(J_r^T J_r)$	30×30 grid, 3 noise levels	59.8
E06	Optimizer shoot-out	3	Success and cost of 5 optimizers	8 random starts, one data set ($\sigma_{\text{frac}} = 0.05$)	161.6
E07	Solver-in-the-loop recovery	2	Solver-induced bias in θ	3 solvers $\times h \in \{0.25, 0.05\} \times \sigma_{\text{frac}} \in \{0, 0.02, 0.1\} \times \Delta t \in \{1, 3\} \times 2$ regimes $\times 10$ reps = 720 fits	92.5
E08	Noise degradation	3	RMSE vs. noise, bootstrap CI	RK4 $h = 0.05$, 4 noise levels $\times 30$ reps	46.8
E09	Sampling & window	3	Effect of Δt and window length	$\Delta t \in \{0.5, 1, 2\} \times 3$ windows $\times 20$ reps	271.2
E10	Initial conditions	3	Unknown I_0 as a third parameter	$I_0 \in \{1, 5, 20\} \times 10$ reps	131.7
E11	Uncertainty quantification	3	Four UQ methods, profile likelihood	60 bootstrap, 60 Monte Carlo, 4×200 MCMC	186.7
E12	Model variants	4	Orders on SEIR, SIRS, SIR-V	4 models $\times 3$ solvers $\times 4$ steps; RK45 as truth	0.6
E13	Crossover sigma*	2+3	Crossover $\sigma^*(h)$ for Euler	$h \in \{0.1, 0.25, 0.5\} \times (3 \text{ bias fits} + 6 \text{ noise levels} \times 25 \text{ reps}) = 459$ fits	50.6
E14	Real data	4	Fit to the Eyam 1666 outbreak	RK4 and Euler ($h = 0.1$), 8 counts	2.2

Table 3: Measured call counts for one fit (E13, Euler, $h = 0.1$, $\sigma_{\text{frac}} = 0.05$).

Stage	Numerical method	Calls
Gold std.	Safeguarded Newton (final size + 60 days)	61
	Adaptive Simpson: table of $t(R)$	2,000
	Adaptive Simpson: refinement on each day	333
Fitting	Levenberg–Marquardt iterations	6
	LU factorizations (steps + covariance)	7
	Euler solves of SIR (residuals, trial costs)	13
	Euler solves of the 9-variable sensitivity system	6

Table 4: Observed convergence order $\hat{p}$ against the gold standard (E01; $h \in \{1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}\}$; error at $t = 40$).

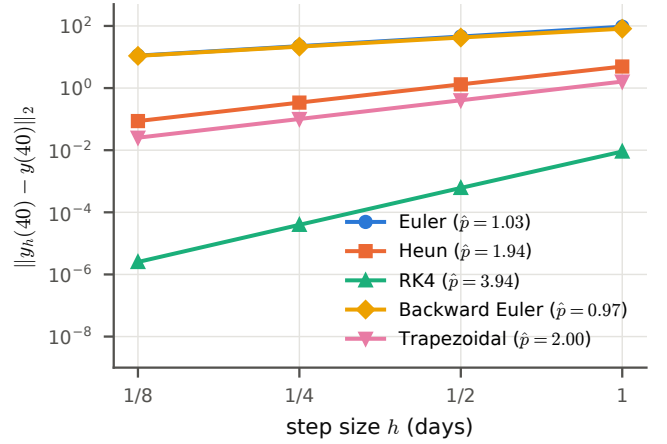
Solver	theory	$\mathcal{R}_0 = 1.5$	$\mathcal{R}_0 = 3$	$\mathcal{R}_0 = 6$
Forward Euler	1	0.98	1.03	1.01
Heun	2	1.99	1.94	1.90
Classical RK4	4	3.98	3.94	3.85
Backward Euler	1	1.02	0.97	1.01
Trapezoidal	2	2.00	2.00	2.00

4.1 RQ1: solver comparison

Convergence order (E01). Table 4 gives the slope $\hat{p}$ of a least-squares fit to $(\ln h, \ln \|y_h(40) - y(40)\|_2)$, measured against the gold standard in three regimes. All 15 estimates lie within 0.15 of theory, and 12 lie within 0.06. The largest deviation is RK4 at $\mathcal{R}_0 = 6$ ($\hat{p} = 3.85$): that epidemic peaks at day 17, so the coarsest step ($h = 1$) is still outside the asymptotic range. Figure 3 shows the reference regime. At the same cost of 640 function evaluations, RK4 with $h = \frac{1}{4}$ has an error of 4×10^{-5} , while Euler with $h = \frac{1}{16}$ would have about 5.5. A higher order buys accuracy far more cheaply than a smaller step.

Which accuracy checks can be trusted (E02). Figure 4 compares the two invariant checks:

- **Mass error.** $\max_t |S + I + R - N|$ stays below 1.2×10^{-12} for every solver and step size, including Euler at $h = 2$, which misplaces the peak by more than five days.

**Figure 3: Global error against the gold standard in the reference regime (E01). Each slope matches its method’s order.**

- **Phase-invariant drift.** $\max_t |Q(t) - Q(0)|$ ranks the solvers correctly and shrinks at each one’s order. From $h = 2$ to $h = 0.1$ it falls from 0.19 to 8.1×10^{-3} for Euler, from 8.2×10^{-3} to 2.0×10^{-5} for Heun, and from 1.2×10^{-5} to 7.2×10^{-11} for RK4.

A modeller who checks $S + I + R = N$ gets a green light on a wrong solution. The invariant $Q(t)$ costs nothing extra and does detect the error.

Stability (E03). Over a 15×15 grid with $(\beta, \gamma) \in [0.05, 1.5] \times [0.05, 1]$, we recorded the largest tested step at which each solver’s solution stayed non-negative and bounded. The median of that step was 2, 4 and 8 days for Euler, Heun and RK4. At that frontier, the largest value of $h\rho(\partial f/\partial y)$ (from our power method) was 2.07, 1.90 and 2.71, against the theoretical real-axis limits of 2, 2 and 2.785 (§2.2). The empirical frontier therefore matches linear stability theory. For the step sizes used in fitting ($h \leq 0.5$), no solver was ever

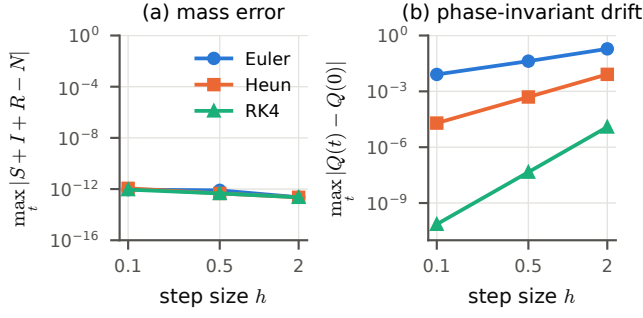


Figure 4: (a) Mass conservation holds to round-off at every h , so it cannot detect a wrong solution. (b) Drift in the phase invariant $Q = \ln S + kR$ separates the solvers by order (E02).

Table 5: Noise-free bias in $\hat{\beta}$ at $(\beta, \gamma) = (0.3, 0.1)$ and $\Delta t = 1$ (E07), and its reduction when h shrinks 5 $\times$.

Solver	$h = 0.25$	$h = 0.05$	ratio	theory
Euler	4.48×10^{-3}	8.93×10^{-4}	5.0	5
Heun	5.88×10^{-5}	2.43×10^{-6}	24.2	25
RK4	7.17×10^{-9}	1.18×10^{-11}	608	625

unstable anywhere on the grid, so every difference below comes from accuracy, not stability.

Epidemiological quantities (E04). At $h = 0.25$, the peak height is off by 2.91 people for Euler, 0.035 for Heun and 0.015 for RK4. The final size S_∞ is off by 1.52, 0.009 and 2×10^{-5} . Heun and RK4 give identical peak-time errors at every h (0.36, 0.36, 0.14 and 0.11 days), because the error comes from where the grid falls, not from the solver. Reading t_{peak} off the solver grid limits its accuracy to about $h/2$.

Answer to RQ1. The three explicit solvers reach their theoretical orders and stability limits on SIR. Below the stability limit, their difference is purely one of accuracy per unit cost, and the phase invariant, not mass conservation, is the check that reveals it.

4.2 RQ2: parameter recovery

Fitting *noise-free* gold-standard data isolates truncation error: a perfect solver would return θ_{true} exactly. The recovery experiment (E07, Figure 5 and Table 5) shows that the bias in $\hat{\beta}$ is not only non-zero but follows each solver's order. Cutting h by a factor of 5, from 0.25 to 0.05, reduces Euler's bias by 5.0 $\times$, Heun's by 24.2 $\times$ and RK4's by 608 $\times$, against the theoretical 5, 25 and 625. The second regime, $(\beta, \gamma) = (0.6, 0.15)$, gives 5.1, 23.5 and 583.

Three further observations follow.

- **The bias is systematic.** All three solvers *overestimate* both parameters. Euler at $h = 0.25$ is 1.5% high in β and 1.4% high in γ , and at $h = 1$ it is 6% high in β ($\hat{\beta} = 0.318$, $\hat{\gamma} = 0.106$).
- **$\mathcal{R}_0$ is much less affected.** At $\mathcal{R}_0 = 3$, Euler at $h = 0.25$ is only 0.10% off in $\hat{\mathcal{R}}_0$, and at $h = 1$ only 0.27%. The bias vector $(4.48, 1.40) \times 10^{-3}$ points 17° from the β axis, almost exactly along the line of constant $\mathcal{R}_0$ through the truth (18°), so the solver moves the estimate along that line and the errors in

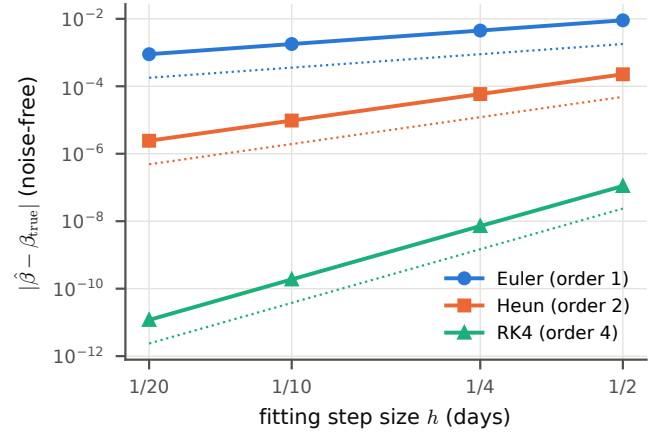


Figure 5: Bias of $\hat{\beta}$ when fitting exact (noise-free) data, against the fitting solver's step size. Dotted guides show slopes h^1 , h^2 and h^4 .

β and γ cancel in their ratio. The cancellation is weaker at $(0.6, 0.15)$, where Euler at $h = 0.25$ is 3.0% off in β , 1.8% off in γ , and 1.2% off in $\mathcal{R}_0$.

- **The data hardly matter.** Changing the sampling interval from $\Delta t = 1$ to 3 changes the bias by less than 2%. The bias belongs to the forward model, not to the data. Because the Jacobian comes from sensitivities integrated by the *same* solver, the fit converges to the minimizer of the *discretized* problem, and that is how truncation error reaches $\hat{\theta}$.

With noise. Once noise is added, how much of the error the solver contributes depends on the noise level. At $h = 0.25$ and $\sigma_{\text{frac}} = 0.02$, Euler's $\text{RMSE}(\hat{\beta})$ is 4.8×10^{-3} , 8 $\times$ that of RK4 (5.9×10^{-4}), and nearly all of Euler's error is bias. At $\sigma_{\text{frac}} = 0.1$ the two are 5.9×10^{-3} and 3.5×10^{-3} , only 1.7 $\times$ apart, because both are dominated by spread. The crossover experiment (§4.4) quantifies this narrowing gap.

Answer to RQ2. The solver's truncation error carries into $(\hat{\beta}, \hat{\gamma})$ with the solver's own order. It is a systematic bias, partly cancelled in $\hat{\mathcal{R}}_0$.

4.3 RQ3: noise and sampling robustness

These experiments fit with RK4 at $h \leq 0.1$, where the solver bias is below 10^{-8} , so what remains is the statistics of the inverse problem.

Cost landscape and optimizers (E05, E06). Near the truth, the curvature eigenvalues of the cost (of $J_r^T J_r$, from the power method with deflation) are 1.21×10^8 and 2.38×10^7 . That gives $\kappa_2(J_r^T J_r) = 5.1$, with the shallow valley along $(0.86, 0.51)$ in the (β, γ) plane. This is the same order as the base paper's $\kappa = 5.2$ for a (β, γ) fit at $\mathcal{R}_0 = 3$ [2]. We started each optimizer from 8 random points in $[0.05, 1] \times [0.02, 0.5]$ and counted a run as a success if it met its own stopping test and ended within 2% of the truth:

- Levenberg–Marquardt succeeded every time, in 8.5 iterations and 19.8 function evaluations on average;
- Nelder–Mead also always succeeded, but needed 111 evaluations;

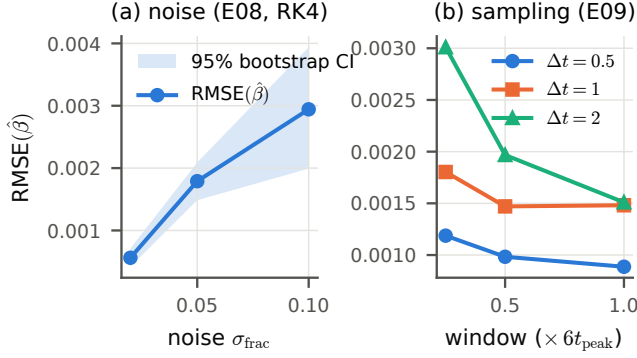


Figure 6: (a) $\text{RMSE}(\hat{\beta})$ grows linearly with noise (E08; 30 replicates; bootstrap CI). (b) RMSE against observation window and sampling interval (E09; 20 replicates per cell).

- Gauss–Newton failed from 2 of 8 starts in linear coordinates, and from 1 of 8 in log coordinates;
- golden-section coordinate descent ended within 2% of the truth from every start, but never met its stopping test within its 10 sweeps. From half the starts it was still short of the least-squares minimum, moving along the tilted valley one axis at a time.

This is why Levenberg–Marquardt is the estimator everywhere else.

Noise (E08). With RK4 at $h = 0.05$, $\text{RMSE}(\hat{\beta})$ is 1.2×10^{-11} on noise-free data, which is the solver floor. It rises to 5.6×10^{-4} , 1.8×10^{-3} and 2.9×10^{-3} at $\sigma_{\text{frac}} = 0.02, 0.05$ and 0.1 (Figure 6a). The growth is nearly linear, $\text{RMSE} \approx 0.03 \sigma_{\text{frac}}$, which supports the $s \approx K\sigma$ assumption behind (11).

Sampling frequency and window (E09). Figure 6b varies the sampling interval Δt and the window length, measured as a fraction of $6t_{\text{peak}}$, at $\sigma_{\text{frac}} = 0.05$. Two findings agree with the base paper:

- **Data deep in the tail add little.** At $\Delta t = 1$, doubling the window from 0.5 to 1.0 changes $\text{RMSE}(\hat{\beta})$ by less than 1%.
- **Where the points fall matters more than how many there are.** The same 115 observations give 1.19×10^{-3} when taken densely around the peak ($\Delta t = 0.5$, short window), but 1.51×10^{-3} when spread over the tail ($\Delta t = 2$, full window).

Coarse sampling and a short window compound each other: at $\Delta t = 2$, the shortest window doubles the error, to 3.0×10^{-3} .

Initial conditions (E10). Treating I_0 as a third unknown raises the condition number of the residual Jacobian from $\kappa_2 \approx 2.2$ – 2.6 to 211, 671 and 1,828 at $I_0 = 1, 5$ and 20 . The variance of $\hat{\beta}$ grows by $18.8\times$, $13.3\times$ and $3.5\times$, and that of $\hat{\gamma}$ by $4.4\times$, $4.3\times$ and $1.3\times$ (10 replicates per cell). The direction matches the base paper: an unknown initial condition badly worsens conditioning. The variance inflation does *not*, however, track κ_2 . The condition number bounds the worst-case amplification, while the variance depends on how the noise projects onto the poorly determined direction.

Uncertainty quantification (E11). On one data set ($\sigma_{\text{frac}} = 0.05$, 60 daily points), the four methods agree closely (Figure 7a):

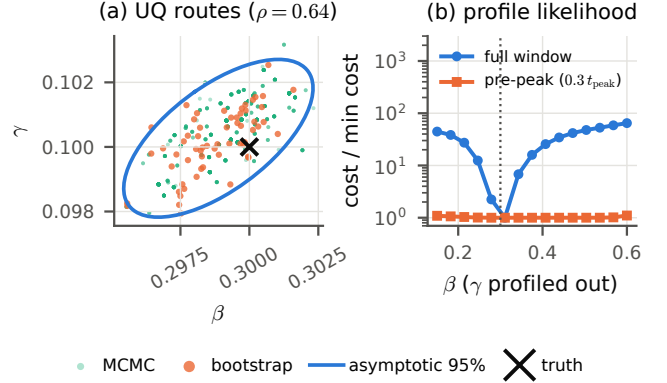


Figure 7: (a) Asymptotic ellipse, bootstrap cloud and MCMC samples for one data set (E11). (b) Profile likelihood on the full window and on a pre-peak window; the flat curve signals non-identifiability.

- **Standard error of $\hat{\beta}$:** 1.40×10^{-3} (asymptotic), 1.33×10^{-3} (bootstrap, 60 resamples) and 1.38×10^{-3} (MCMC).
- **Monte Carlo:** over 60 fresh data sets the spread is 1.80×10^{-3} . The measured bias, -2.2×10^{-4} , cannot be told apart from zero with 60 samples.
- **Correlation:** $\rho(\hat{\beta}, \hat{\gamma})$ is 0.64, 0.67 and 0.75 from the first three methods.

The MCMC chains are short in the quick profile (4 chains of 200 samples). Their acceptance rate is 0.17–0.22, $\hat{R} = 1.09$ and 1.05, and the effective sample size is about 35–40, so the MCMC numbers are indicative only. The profile likelihood (Figure 7b) shows the identifiability collapse predicted by the base paper. On the full window the profiled cost rises 65 $\times$ across $\beta \in [0.15, 0.6]$. When the data stop at $0.3 t_{\text{peak}}$ (day 11.5, during exponential growth), it varies by only 10%: early-growth data constrain the growth rate $\beta - \gamma$, but not β and γ separately.

Range of true (β, γ) . The convergence results hold in all three regimes (Table 4), stability holds over the whole 15×15 grid, and the order scaling of the bias holds in both recovery regimes. The size of the bias does depend on the regime. At $(0.6, 0.15)$, Euler’s bias in $\hat{\beta}$ at $h = 0.25$ is $4\times$ larger (1.82×10^{-2}) than at $(0.3, 0.1)$, because the faster epidemic is harder to integrate.

Answer to RQ3. With an accurate solver, the window, the sampling design and unknown initial conditions dominate the uncertainty, exactly as Capaldi et al. found. The four uncertainty methods agree, and the error grows linearly with noise.

4.4 Where RQ2 meets RQ3: the crossover σ^*

The crossover experiment (E13) fits an Euler-based model to gold-standard data at $h \in \{0.1, 0.25, 0.5\}$ and 7 noise levels, with 25 replicates per noisy cell. Figure 8 overlays the bias $|b(h)|$ (dashed) on the noise spread $s(\sigma, h)$ (solid). Two facts make the decomposition (10) clean:

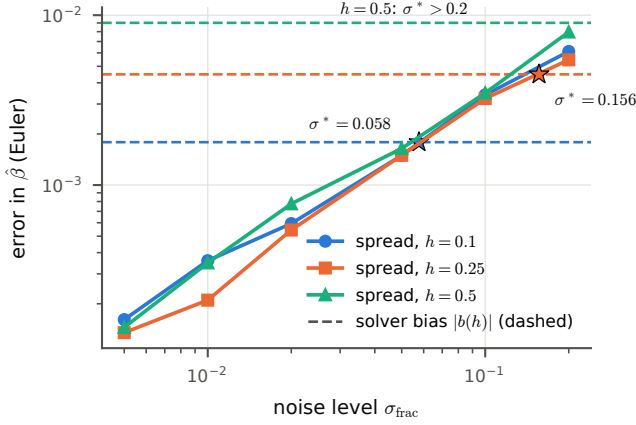


Figure 8: The crossover for Euler (E13). Dashed lines: the solver bias at each h . Solid curves: the Monte Carlo spread over 25 noisy data sets. Stars mark σ^* , where the two are equal. Left of a star the solver dominates the error in $\hat{\beta}$; right of it, noise does.

Table 6: Crossover noise levels σ^* for Euler (E13), as a fraction of peak prevalence. “Predicted” is $|b|/K$, with $K = 0.033$ the least-squares slope of the spread against σ .

h	$ b_\beta(h) $	σ_β^*	predicted	$ b_\gamma(h) $	σ_γ^*
0.10	1.79×10^{-3}	0.058	0.055	5.5×10^{-4}	0.029
0.25	4.48×10^{-3}	0.156	0.137	1.4×10^{-3}	0.054
0.50	9.02×10^{-3}	> 0.2	0.28	2.8×10^{-3}	0.128

- **The bias depends only on h .** It grows $1.79 \rightarrow 4.48 \rightarrow 9.02 \times 10^{-3}$, ratios of 2.51 and 2.01 for step ratios of 2.5 and 2, as an order-1 method should.
- **The spread depends only on σ .** At $\sigma_{\text{frac}} = 0.05$ it is 1.50, 1.50 and 1.64×10^{-3} for the three step sizes.

Table 6 lists the crossovers. For $\hat{\beta}$, $\sigma^* = 0.058$ at $h = 0.1$ and 0.156 at $h = 0.25$. At $h = 0.5$ the bias still exceeds the spread at the largest noise level tested ($\sigma_{\text{frac}} = 0.2$), so there $\sigma^* > 0.2$.

The scaling law (11) predicts these values well. A least-squares fit of $s = K\sigma$ through the origin, over all 18 noisy cells, gives $K = 0.033$, and $\sigma^* \approx |b(h)|/K$ then predicts 0.055, 0.137 and 0.28. These are within 5% and 12% of the first two measured values, and consistent with the bound on the third. For $\hat{\gamma}$ the crossover comes earlier (0.029, 0.054 and 0.128), because γ 's bias is smaller relative to its spread.

The same law, applied to the noise-free biases of Figure 5, gives a rough guide for the other solvers. At $h = 0.1$, Heun's bias (9.6×10^{-6}) puts σ^* near 3×10^{-4} , and RK4's bias (1.9×10^{-10}) puts it near 6×10^{-9} . For RK4, therefore, the solver never matters at any realistic noise level. For Euler it matters whenever surveillance noise is below a few percent of the peak. At $\sigma_{\text{frac}} = 0.005$ Euler's total error is about $11 \times$ the noise spread (1.79×10^{-3} against 1.6×10^{-4}), and more data cannot recover it, because bias does not average out.

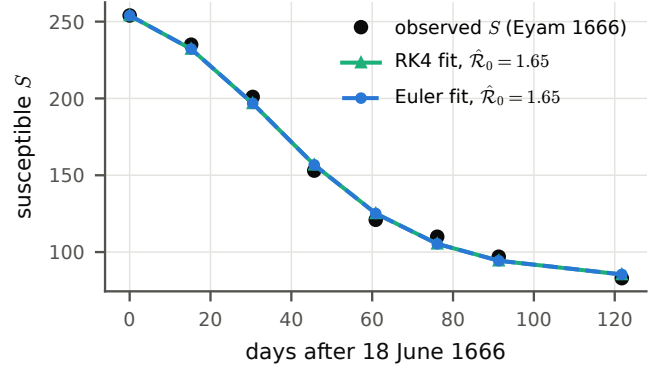


Figure 9: Eyam 1666 susceptible counts [34] with the RK4 and Euler fits (E14), which are visually indistinguishable.

4.5 RQ4: extensions

Model variants (E12). The variants have no closed form, so we measured them against a tight RK45 solve ($\text{rtol} = 10^{-12}$), all at $\mathcal{R}_0 = 3$:

- SEIR, with incubation rate $\sigma_{\text{inc}} = 0.2$;
- SIRS, with waning rate $\xi = 0.02$;
- SIR with vaccination, at rate $\nu = 0.01$.

The observed orders are 1.01, 1.03 and 1.00 for Euler; 1.97, 1.95 and 1.95 for Heun; and 3.99, 3.93 and 3.94 for RK4. The ordering, and with it the mechanism behind RQ2, is not specific to the plain SIR structure.

Eyam 1666 (E14). Fitting $S(t)$ to the 8 susceptible counts (Figure 9) with RK4 gives $\hat{\beta} = 0.147 \pm 0.005$ and $\hat{\gamma} = 0.089 \pm 0.005$ per day, so $\hat{\mathcal{R}}_0 = 1.65$ and a mean infectious period $1/\hat{\gamma}$ of 11 days. Euler gives the same values to three significant figures ($\hat{\mathcal{R}}_0 = 1.646$ against 1.648), a difference of 0.18% in $\hat{\beta}$, or $1/20$ of a standard error. The residuals reach ± 4.6 people (1.8% of N) on only 8 points, so this data set lies far inside the noise-dominated regime, and solver choice is irrelevant there, just as (11) predicts.

Comparison with the original study. Raggett fitted a stochastic SIR model to both the susceptible and the infective counts, and estimated a removal rate $\hat{\alpha} = 3.39$ and an infection rate $\hat{\beta}_R = 0.0212$ per susceptible-infective pair, both per month [13, 34]. In our parameterization ($\gamma = \alpha$, $\beta = \beta_R N$) these are 0.111 and 0.182 per day, so $\mathcal{R}_0 = 1.63$. Our $\hat{\mathcal{R}}_0 = 1.65$ agrees within 1%, while our individual rates are about 20% lower. That is exactly the pattern the correlation predicts: $\rho(\hat{\beta}, \hat{\gamma}) = 0.985$, close to 1, as in the near-threshold case of Capaldi et al. ($\mathcal{R}_0 = 1.2$, $\rho = 0.98$). The data pin down $\mathcal{R}_0$ far better than β or γ alone.

4.6 Comparison with the base paper

Our independent code reproduces the base paper's identifiability findings:

- the two-parameter problem has a condition number of the same order ($\kappa \approx 5$);
- fitting I_0 as well raises κ by two to three orders of magnitude;
- the information is concentrated up to and around the peak, and pre-peak data leave β and γ unidentifiable;

- near threshold the two estimates are strongly correlated (Eyam, $\rho = 0.985$).

One quantitative difference is instructive. In our reference regime $\rho = 0.64$, while Capaldi et al. report $\rho = 0.11$ at $\mathcal{R}_0 = 3$. Their noise model is different ($\xi = \frac{1}{2}$), their population larger ($N = 10^4$, $I_0 = 100$), and above all their window is longer: they observe until I returns to I_0 , while ours ends at $1.56 t_{\text{peak}}$. The correlation is a property of the *design*, not of $\mathcal{R}_0$ alone, which reinforces the base paper’s own message about sampling.

The solver axis is new. The base paper’s adaptive ode45 keeps its truncation error under tolerance control, in all likelihood far below the noise levels studied. By our analysis it was therefore working above σ^* , which justifies the black-box treatment after the fact, but only in that regime. A fixed-step low-order solver, or a loose tolerance, can leave that regime without any visible warning.

4.7 Discussion

The question “does the solver matter?” has a two-sided answer. Both “always use RK4” and “the solver never matters” are wrong as general statements. A low-order solver biases the estimate by an amount set by h^p , and that bias competes with a noise spread set by σ . The boundary $\sigma^* \propto h^p$ can be measured, and a simple linear argument predicts it to within about 12%. For a practitioner this gives a rule: *estimate the noise level, then choose h and p so that $\sigma^*(h)$ lies well below it.* With RK4, any $h \leq 0.5$ day is enough. With Euler, $h = 0.1$ is safe only when the noise exceeds about 6% of the peak.

Common checks can mislead. Mass conservation, the most common sanity check, cannot detect a wrong SIR solution, while the phase invariant $Q(t)$ is free and informative. Likewise, a peak time read off the solver grid is limited by the grid, not by the solver.

Solver error is bias, not noise. It does not average out over replicates or extra observations, and it enters the Jacobian through the sensitivity equations, so better data cannot fix it. This is also why the error budget (10) must be measured against a solver-free truth: a “finer numerical solve” used as the reference would hide part of $b(h)$.

5 Conclusion & Future Work

5.1 Key takeaways

- (1) **RQ1.** Measured against a solver-free gold standard, our hand-written solvers reach their textbook orders: all 15 estimates are within 0.15 of theory. Their stability limits match linear theory. Mass conservation is a useless accuracy check for SIR; the phase invariant is a reliable one.
- (2) **RQ2.** Solver truncation error passes into the fitted parameters with the *same order*. A $5\times$ smaller step reduces the bias in $\hat{\beta}$ by 5.0, 24.2 and 608 for Euler, Heun and RK4. The bias is systematic and is largely cancelled in $\hat{\mathcal{R}}_0$.
- (3) **RQ3.** With an accurate solver, the observation window, the sampling design and unknown initial conditions dominate the uncertainty, reproducing the base paper. Four independent uncertainty methods agree.
- (4) **Crossover.** Solver bias and noise spread separate cleanly, and they are equal at a measurable σ^* . For Euler it is 0.058 at

$h = 0.1$ and 0.156 at $h = 0.25$, in line with $\sigma^* \approx |b(h)|/K \propto h^p$.

- (5) **RQ4.** The ordering holds on SEIR, SIRS and SIR-V. On the Eyam data, which are noise-dominated, solver choice is irrelevant, and $\hat{\mathcal{R}}_0 = 1.65$ agrees with the original study’s estimate within 1%.

All code, data, experiment configurations and results are public at <https://github.com/g-schatten/CSE402-Project>.

5.2 Limitations

- **Profile scale.** All results use the quick profile, with 10–60 replicates per cell. The orderings and scaling laws are already clean, but Monte Carlo quantities such as the initial-condition variance ratios carry visible sampling noise, and the short MCMC chains ($\hat{R} \approx 1.05\text{--}1.09$) are only indicative.
- **σ^* is measured for Euler only.** The Heun and RK4 crossovers are extrapolated from (11), because they lie far below the smallest noise level tested.
- **Synthetic, correctly specified data.** The core results use data generated from the model being fitted, with additive Gaussian noise ($\xi = 0$). This isolates solver error but says nothing about model misspecification.
- **One simple real-data fit.** The Eyam fit uses a deterministic model and only the susceptible counts, and the counts come from published reproductions of Raggett’s table rather than from his paper directly.
- **Small, non-stiff models.** All the models tested are small and non-stiff, so the implicit solvers’ stability advantages never come into play.

5.3 Future work

- Run the full profile (about 1.9×10^6 fits for E07), with bootstrap confidence intervals on σ^* itself.
- Measure σ^* directly for Heun and RK4, by testing much lower noise levels, and for adaptive RK45 as a function of its tolerance.
- Repeat the crossover under Poisson and negative-binomial noise, which are already implemented in `observe.py`, and for incidence data.
- Add model misspecification as a third error source, for example fitting SIR to SEIR-generated data, and map where each of the three sources dominates.
- Fit the Eyam infective counts as well, with a stochastic SIR model as Raggett did, and fit further historical outbreaks.

References

- [1] M. Bostock, V. Ogievetsky, and J. Heer. 2011. D³: Data-Driven Documents. *IEEE Transactions on Visualization and Computer Graphics* 17, 12 (2011), 2301–2309. doi:10.1109/TVCG.2011.185
- [2] A. Capaldi, S. Behrend, B. Berman, J. Smith, J. Wright, and A. L. Lloyd. 2012. Parameter estimation and uncertainty quantification for an epidemic model. *Mathematical Biosciences and Engineering* 9, 3 (2012), 553–576. doi:10.3934/mbe.2012.9.553
- [3] Department of CSE, BUET. 2026. Eigenvalue Decomposition: The Power Method. Lecture slides, CSE 401: Numerical Analysis, Simulation and Modeling, Department of CSE, BUET.
- [4] J. R. Dormand and P. J. Prince. 1980. A Family of Embedded Runge–Kutta Formulae. *J. Comput. Appl. Math.* 6, 1 (1980), 19–26. doi:10.1016/0771-050X(80)90013-3

- [5] B. Efron and R. J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall, New York, NY.
- [6] Framer. 2024. Framer Motion: Animation Library for React. <https://motion.dev>. Version 11.
- [7] A. Gelman and D. B. Rubin. 1992. Inference from Iterative Simulation Using Multiple Sequences. *Statist. Sci.* 7, 4 (1992), 457–472. doi:10.1214/ss/1177011136
- [8] A. Golightly and C. Sherlock. 2019. Efficient Sampling of Conditioned Markov Jump Processes. arXiv:1809.07139. Table 4 reproduces Raggett's Eyam data.
- [9] H. Haario, E. Saksman, and J. Tamminen. 2001. An Adaptive Metropolis Algorithm. *Bernoulli* 7, 2 (2001), 223–242.
- [10] E. Hairer, S. P. Nørsett, and G. Wanner. 1993. *Solving Ordinary Differential Equations I: Nonstiff Problems* (2nd ed.). Springer, Berlin.
- [11] C. R. Harris, K. J. Millman, S. J. van der Walt, et al. 2020. Array programming with NumPy. *Nature* 585 (2020), 357–362. doi:10.1038/s41586-020-2649-2
- [12] H. W. Hethcote. 2000. The Mathematics of Infectious Diseases. *SIAM Rev.* 42, 4 (2000), 599–653. doi:10.1137/S0036144500371907
- [13] L. S. T. Ho, J. Xu, F. W. Crawford, V. N. Minin, and M. A. Suchard. 2017. Birth/Birth-Death Processes and Their Computable Transition Probabilities with Biological Applications. arXiv:1603.03819. Table 1 reproduces Raggett's Eyam data.
- [14] J. D. Hunter. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95. doi:10.1109/MCSE.2007.55
- [15] KaTeX contributors. 2024. KaTeX: Fast Math Typesetting for the Web. <https://katex.org>. Version 0.16.
- [16] W. O. Kermack and A. G. McKendrick. 1927. A Contribution to the Mathematical Theory of Epidemics. *Proceedings of the Royal Society of London. Series A* 115, 772 (1927), 700–721. doi:10.1098/rspa.1927.0118
- [17] T. Kluyver, B. Ragan-Kelley, F. Pérez, et al. 2016. Jupyter Notebooks – a Publishing Format for Reproducible Computational Workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. IOS Press, Amsterdam, 87–90. doi:10.3233/978-1-61499-649-1-87
- [18] H. Krekel et al. 2024. pytest: helps you write better programs. <https://pytest.org>.
- [19] D. Lemire. 2019. Fast Random Integer Generation in an Interval. *ACM Transactions on Modeling and Computer Simulation* 29, 1 (2019), 3:1–3:12. doi:10.1145/3230636
- [20] K. Levenberg. 1944. A Method for the Solution of Certain Non-Linear Problems in Least Squares. *Quart. Appl. Math.* 2, 2 (1944), 164–168.
- [21] D. R. MacIver, Z. Hatfield-Dodds, et al. 2019. Hypothesis: A New Approach to Property-Based Testing. *Journal of Open Source Software* 4, 43 (2019), 1891. doi:10.21105/joss.01891
- [22] D. W. Marquardt. 1963. An Algorithm for Least-Squares Estimation of Nonlinear Parameters. *J. Soc. Indust. Appl. Math.* 11, 2 (1963), 431–441. doi:10.1137/0111030
- [23] G. Marsaglia and W. W. Tsang. 2000. A Simple Method for Generating Gamma Variables. *ACM Trans. Math. Software* 26, 3 (2000), 363–372. doi:10.1145/358407.358414
- [24] G. Marsaglia and W. W. Tsang. 2000. The Ziggurat Method for Generating Random Variables. *Journal of Statistical Software* 5, 8 (2000), 1–7. doi:10.18637/jss.v005.i08
- [25] W. McKinney. 2010. Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference*. SciPy, Austin, TX, 56–61. doi:10.25080/Majora-92bf1922-00a
- [26] Meta Open Source. 2024. React: The Library for Web and Native User Interfaces. <https://react.dev>. Version 18.3.
- [27] Microsoft. 2024. TypeScript: JavaScript with Syntax for Types. <https://www.typescriptlang.org>. Version 5.6.
- [28] M. E. O'Neill. 2014. *PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation*. Technical Report HMC-CS-2014-0905. Harvey Mudd College.
- [29] Poimandres. 2024. Zustand: State Management for React. <https://github.com/pmndrs/zustand>. Version 4.5.
- [30] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery. 2007. *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). Cambridge University Press, Cambridge.
- [31] D. Prodanov. 2021. Analytical Parameter Estimation of the SIR Epidemic Model. Applications to the COVID-19 Pandemic. *Entropy* 23, 1 (2021), 59. doi:10.3390/e23010059
- [32] Python Software Foundation. 2022. The Python Language Reference, version 3.11. <https://docs.python.org/3.11/reference/>.
- [33] PyYAML contributors. 2024. PyYAML: YAML Parser and Emitter for Python. <https://pyyaml.org>. Version 6.
- [34] G. Raggett. 1982. A Stochastic Model of the Eyam Plague. *Journal of Applied Statistics* 9, 2 (1982), 212–225.
- [35] Remix Software. 2024. React Router. <https://reactrouter.com>. Version 6.26.
- [36] B. Veytsman. 2026. acmart: Class for Typesetting Publications of ACM. <https://ctan.org/pkg/acmart>. Version 2.19.
- [37] P. Virtanen, R. Gommers, T. E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272. doi:10.1038/s41592-019-0686-2
- [38] Vite contributors. 2024. Vite: Frontend Build Tool. <https://vite.dev>. Version 5.4.
- [39] Vitest contributors. 2024. Vitest: Unit Testing Framework Powered by Vite. <https://vitest.dev>. Version 2.1.
- [40] S. U. Zaman, S. M. Masfi, I. B. Ahmed, S. N. Raiyan, and A. Chowdhury. 2026. SIRLab: Numerical Analysis of the SIR Epidemic Model (source code). <https://github.com/g-schatten/CSE402-Project>.